

TEMA 1. Clases y objetos

1. ¿Cuáles son las cuatro características básicas de la programación orientada a objetos? Describe brevemente cada una.

Respuesta:

Abstracción: Capacidad de aislar las características esenciales de un objeto, ignorando los detalles no relevantes para el problema actual.

Encapsulamiento: Agrupación de datos y métodos en una unidad (la clase), protegiendo el acceso a la información mediante modificadores de visibilidad.

Herencia: Permite que una clase nueva (hija) herede atributos y comportamientos de una clase existente (padre), facilitando la reutilización de código.

Polimorfismo: Capacidad de que objetos de diferentes tipos respondan de forma distinta al mismo mensaje o llamada a un método.

2. Cita cuatro lenguajes populares que permitan la programación orientada a objetos:

Respuesta:

Java

C++

Python

C#

3. Los paradigmas anteriores a la POO, ¿Qué es la programación estructurada? y, todavía mejor, ¿Qué es la programación modular?

Respuesta:

Programación Estructurada: Se basa en dividir el flujo del programa en estructuras de control (secuencias, bucles y condiciones) y funciones, evitando el uso de "saltos" (goto).

Programación Modular: Va un paso más allá, dividiendo el programa en bloques independientes llamados módulos. Cada módulo tiene una responsabilidad clara y se comunica con los demás a través de interfaces, facilitando el mantenimiento.

4. ¿Qué tres elementos definen a un objeto en programación orientada a objetos?

Respuesta

Estado: Sus datos o atributos (lo que el objeto sabe).

Comportamiento: Sus métodos o funciones (lo que el objeto puede hacer).

Identidad: Lo que lo distingue de otros objetos (aunque tengan el mismo estado).

5. ¿Qué es una clase? ¿Es lo mismo que un objeto? ¿Qué es una instancia? ¿Todos los lenguajes orientados a objetos manejan el concepto de clase?

Respuesta

Clase: Es el "molde" o plantilla que define qué atributos y métodos tendrán los objetos.

Objeto: Es la entidad creada a partir de la clase.

Instancia: Es un sinónimo de objeto. Decimos que "un objeto es una instancia de una clase".

¿Todos los lenguajes? No todos. Algunos, como JavaScript (en sus versiones originales), usan prototipos en lugar de clases formales.

6. ¿Dónde se almacenan en memoria los objetos? ¿Es igual en todos los lenguajes? ¿Qué es la recolección de basura?

Respuesta

Memoria: Generalmente, los objetos **se almacenan en el Heap** en la mayoría de lenguajes. Las variables locales que referencian a esos objetos están en el Stack (pila).

Diferencias: Sí, **lenguajes como C++ permiten crear objetos tanto en Stack como en Heap**; Java obliga a que casi todos los objetos vivan en el Heap.

Ventajas del uso del heap:

- Reservo dinámicamente, el tamaño se decide en ejecución.
- Lo que está en el heap, vive más allá que el método o función donde se ha creado.

Desventajas:

- Hay que liberarla cuando ya no se necesita. O bien de forma manual(difícil de hacer) o bien de forma automática(por ejemplo, con un recolector de basura).

Recolección de basura (Garbage Collection): Es un proceso automático que libera la memoria ocupada por objetos que ya no tienen ninguna referencia activa, evitando fugas de memoria. Los memory leaks si existen en java, ya que a veces hay referencias que realmente son inútiles para el programador pero el recolector de basura no lo elimina porque está siendo referenciado.

La desventaja del recolector de basura es que a veces provoca un ligero problema de rendimiento porque no siempre hay basura que eliminar, provocando sobrecarga. En la industria se usa igual porque las ventajas que producen son mayores que las desventajas.

7. ¿Qué es un método? ¿Qué es la sobrecarga de métodos?

Respuesta

Método: Una función definida dentro de una clase que opera sobre los datos del objeto.

Sobrecarga: Es tener varios métodos con el mismo nombre pero con diferente firma (distinto número o tipo de parámetros) dentro de la misma clase.

8. Ejemplo mínimo de clase en Java, que se llame Punto, con dos atributos, x e y, con un método que se llame calculaDistanciaAOrigen, que calcule la distancia a la posición 0,0. Por sencillez, los atributos deben tener visibilidad por defecto. Crea además un ejemplo de uso con una instancia y uso del método

Respuesta

```
public class Punto {  
    double x, y; // Visibilidad por defecto  
  
    double calculaDistanciaAOrigen() {  
        return Math.sqrt(x * x + y * y);  
    }  
}  
  
// Ejemplo de uso  
public class Principal {  
    public static void main(String[] args) {  
        Punto p1 = new Punto();  
        p1.x = 3;  
        p1.y = 4;  
        System.out.println("Distancia: " + p1.calculaDistanciaAOrigen());  
    }  
}
```

9. ¿Cuál es el punto de entrada en un programa en Java? ¿Qué es static y para qué vale? ¿Sólo se emplea para ese método main? ¿Para qué se combina con final?

Respuesta

Punto de entrada: El método public static void main(String[] args).

static: Significa que el miembro pertenece a la clase y no a una instancia específica. Se puede usar sin crear un objeto. No existe this. No puedo usar desde un método static nada que no sea static. No debemos abusar del uso de static

¿Solo en main? No, se usa para constantes, métodos de utilidad (como Math.sqrt()) o contadores globales.

static final: Se usa para definir constantes que pertenecen a la clase y cuyo valor no puede cambiar.

10. Intenta ejecutar un poco de Java de forma básica, con los comandos javac y java. ¿Cómo podemos compilar el programa y ejecutarlo desde línea de comandos? ¿Java es compilado? ¿Qué es la máquina virtual? ¿Qué es el byte-code y los ficheros .class?

Respuesta

Comandos:

Compilar: javac Punto.java (genera Punto.class).

Ejecutar: java Punto.

¿Es compilado? Java es híbrido. Se compila a un lenguaje intermedio (Bytecode) y luego la JVM (Máquina Virtual de Java) lo interpreta o compila en tiempo de ejecución (JIT) a código máquina.

Ficheros .class: Contienen el Bytecode, que es multiplataforma.

11. En el código anterior de la clase Punto ¿Qué es new? ¿Qué es un constructor? Pon un ejemplo de constructor en una clase Empleado que tenga DNI, nombre y apellidos

Respuesta

new: Operador que reserva memoria en el Heap para un nuevo objeto y llama al constructor.

Varias cosas:

- 1.- Reserva memoria
- 2.- Invoca constructor
- 3.- Es una expresión (puedo asignarla a una variable, o usarla directamente en línea)

Constructor: Método especial que se ejecuta al crear el objeto para inicializar su estado.

Java

```

public class Empleado {
    String dni, nombre, apellidos;

    public Empleado(String dni, String nombre, String apellidos) {
        this.dni = dni;
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
}

```

12. ¿Qué es la referencia this? ¿Se llama igual en todos los lenguajes? Pon un ejemplo del uso de this en la clase Punto

Respuesta

this: Es una referencia al objeto actual (el que está ejecutando el método). Sirve para desambiguar/aclarar. No está disponible en métodos static. En otros lenguajes puede tener otro nombre.

Nombres: En Python se llama self, en PHP \$this, en Visual Basic Me.

Ejemplo: this.x = x; sirve para diferenciar el atributo de la clase del parámetro del constructor si se llaman igual.

13. Añade ahora otro nuevo método que se llame distanciaA, que reciba un Punto como parámetro y calcule la distancia entre this y el punto proporcionado

Respuesta

```

double distanciaA(Punto otro) {
    double dx = this.x - otro.x;
    double dy = this.y - otro.y;
    return Math.sqrt(dx * dx + dy * dy);
}

```

14. El paso del Punto como parámetro a un método, es por copia o por referencia, es decir, si se cambia el valor de algún atributo del punto pasado como parámetro, dichos cambios afectan al objeto fuera del método? ¿Qué ocurre si en vez de un Punto, se recibiese un entero (int) y dicho entero se modificase dentro de la función?

Respuesta

Objetos (Punto): En Java se pasa la referencia por valor. Si modificas un atributo del objeto dentro del método, el cambio sí afecta al objeto original.

Primitivos (int): Se pasan por valor. Se crea una copia, por lo que cualquier cambio interno no afecta a la variable original.

15. ¿Qué es el método toString() en Java? ¿Existe en otros lenguajes? Pon un ejemplo de toString() en la clase Punto en Java

Respuesta

Es un método de la clase Object que devuelve una representación en texto del objeto. Se suele sobreescribir para dar información útil.

En otros lenguajes: En Python es `__str__`, en C# es `ToString()`.

Java

@Override

```
public String toString() {  
    return "(" + x + ", " + y + ")";  
}
```

16. Reflexiona: ¿una clase es como un struct en C? ¿Qué le falta al struct para ser como una clase y las variables de ese tipo ser instancias?

Respuesta

Una clase es conceptualmente similar a un struct de C, pero a un struct le falta:

Métodos: El struct solo guarda datos.

Encapsulamiento: No tiene mecanismos nativos de visibilidad (public/private).

Herencia.

17. Quidemos un poco de magia a todo esto: ¿Como se podría “emular”, con struct en C, la clase Punto, con su función para calcular la distancia al origen? ¿Qué ha pasado con this?

Respuesta

Para emular la clase Punto en C, usaríamos una estructura y una función que reciba un puntero a esa estructura:

C

```
struct Punto {  
    double x, y;  
};
```

```
double calculaDistanciaAOrigen(struct Punto *p) {  
    return sqrt(p->x * p->x + p->y * p->y);  
}
```

¿Qué pasó con this? En C, el this no es automático. Tienes que pasarlo explícitamente como el parámetro struct Punto *p para que la función sepa sobre qué datos trabajar.